Q-learning 学习笔记

这篇笔记主要是给自己复习用，重点把下面几件事讲清楚：

- `Q(s, a)` 到底是什么
- Q 表为什么会有一种“价值从后往前传递”的感觉
- 更新公式里为什么一定要看 `next_state`
- `alpha`、`gamma`、`epsilon` 分别在控制什么

1. Q-learning 在学什么

Q-learning 的核心目标不是直接学“哪个状态好”，而是学习：

```
在状态 s 下，如果我采取动作 a，未来大概能拿到多少累计回报
```

这就是动作价值函数，也就是：

```
Q(s, a)
```

所以我现在对 `Q(s, a)` 的理解应该是：

- 它不是当前这一步的即时奖励
- 它是“当前先做这个动作，然后未来继续尽量做对”的总收益估计

2. reward 和 Q 值的区别

这两个概念很容易混：

- `reward`：当前这一步执行动作后，环境立刻给我的反馈
- `Q(s, a)`：从当前动作出发，一直到后面结束为止，未来累计回报的估计值

可以理解成：

- `reward` 只看眼前
- `Q` 看的是“这一步之后整条路大概值多少钱”

例如在 FrozenLake 里：

- 大多数时候，走一步的 `reward = 0`

- 但某些动作的 `Q` 依然可以很高

原因是：

虽然这一步没奖励，但它可能把我带到了一个“离终点更近”的状态，所以未来总收益更高。

3. Q 表是什么

Q 表本质上就是一个表格：

- 行表示状态 `state`
- 列表示动作 `action`
- 每个格子里存的是 `Q(s, a)`

在你的 `FrozenLake.py` 里：

- `state_size = env.observation_space.n`
- `action_size = env.action_space.n`
- `Q = np.zeros((state_size, action_size))`

这说明：

- 每个状态都对应若干动作
- 每个 `(状态, 动作)` 对应一个估计价值

训练的过程，本质上就是不断把这张表填得更靠谱。

4. 更新公式

Q-learning 的核心更新公式是：

```
Q(s,a) = Q(s,a) + alpha * (reward + gamma * max_a' Q(s',a') - Q(s,a))
```

也可以拆成：

```
新值 = 旧值 + 学习率 * (目标值 - 旧值)
```

其中：

- `Q(s,a)`：当前这对 `(状态, 动作)` 原来的估计

- `alpha`：学习率，决定这次更新要“信多少”
- `reward`：当前动作带来的即时奖励
- `gamma`：折扣因子，决定未来奖励看得多重
- `max Q(s', a')`：到达下一状态后，未来最好的收益估计

所以它真正想表达的是：

```
当前动作值 = 当前奖励 + 未来最优收益
```

更准确地说，是：

```
当前动作值 朝着 当前奖励 + 未来最优收益 这个目标去靠近
```

5. 为什么一定要看 next_state

这一点是 Q-learning 最核心的地方之一。

当我在状态 `s` 做了动作 `a` 后，真正发生的是：

1. 我先得到一个即时奖励 `reward`
2. 我进入下一个状态 `s'`
3. 从 `s'` 开始，后面还会继续产生未来收益

所以当前这一步的价值，不可能只由当前状态决定，而必须由下面两部分组成：

- 眼前拿到多少奖励
- 进入下一个状态后，未来还能赚多少

因此更新时必须看：

```
max Q(s', a')
```

而不是看当前状态里别的动作。

因为你已经执行完当前动作了，接下来能决定未来收益的，是“到达下一个状态以后还能怎么走”。

6. 为什么会感觉价值是“从后往前传递”的

你这个直觉是对的，而且这是理解 Q-learning 的关键。

Q 表初始全是 0，所以一开始谁也不知道哪条路更好。

只有当智能体真的接近终点、甚至到达终点时，价值信息才第一次出现。

例如：

- 某一步到达终点，拿到 `reward = 1`
- 那么终点前一格对应的某个动作，Q 值就会先升高
- 下一次再访问更前面的状态时，它就能看到“后面那个状态已经更值钱了”
- 于是价值继续往前传

所以虽然智能体在环境里是一步一步向前走，
但 Q 值的改善看起来像是从终点附近开始，一层层往前倒推。

这也是你说的“更新顺序像是从后向前”的来源。

7. 参数怎么理解

7.1 alpha

`alpha` 是学习率，控制“新经验对旧估计改动多大”。

- 太大：更新更激进，学得快，但容易震荡
- 太小：更新更保守，更平滑，但学习慢

你原来的理解“表示对更新的重要性的判断”是对的，但可以进一步说得更准确一些：

`alpha` 不是判断这次更新“对不对”，而是控制“这次新信息占多大权重”。

7.2 gamma

`gamma` 是折扣因子，控制智能体有多看重未来。

- `gamma = 0`：只看眼前奖励，不考虑未来
- `gamma` 接近 `1`：非常看重长期收益

这里要纠正一个点：

`gamma = 0` 不等于“没学”。

更准确地说：

- 它仍然在学
- 只是学成了“只关心即时奖励”的短视策略

在 FrozenLake 这种“终点奖励在后面才出现”的任务里，如果 `gamma = 0` ，通常很难学到好策略，因为前面大多数步的即时奖励都是 0。

7.3 epsilon

`epsilon` 控制探索概率。

在你的代码里：

- 以 `epsilon` 的概率随机选动作
- 否则选当前 Q 表里最优动作

这就是 `epsilon-greedy` 。

它解决的问题是：

- 如果一开始就永远贪心
- 由于 Q 表初始几乎全一样
- 智能体可能一直重复某些无效动作
- 很多状态和动作根本没机会被尝试到

需要纠正一个说法：

“不探索就完全不会发生更新” 这句话不准确。

更准确地说：

- 不探索，仍然会更新
- 但更新会局限在很少一部分已走过的路径上
- 结果容易陷入次优策略，甚至一直碰不到真正好的路径

8. 为什么 FrozenLake 适合学 Q-learning

你现在用的是：

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

这里 `is_slippery=False` 的意义不只是“新手友好”，它还有一个很重要的强化学习含义：

- 环境变成了确定性的
- 你执行某个动作后，结果更可预测

- 同一个 `(state, action)` 更稳定地对应某个后果

这会让 Q-learning 更容易收敛，也更方便观察 Q 值传播。

如果改成 `is_slippery=True`：

- 环境会带随机性
- 你想往右，实际可能滑去别处
- 同一个动作的后果不再总是一样

这样训练会更难，Q 值更新也会更“平均化”，需要更多回合才能学得稳定。

9. 为什么 Q 表从 0 开始也能学起来

初始全 0 不影响学习，因为：

- 智能体会探索
- 探索过程中会真的走到终点或失败状态
- 一旦某些位置第一次收到有效反馈，Q 值就开始变化
- 这些变化再继续传回更早的位置

这里也要纠正一个点：

“初值太大没影响，太小也没啥影响” 这个说法不准确。

初始值是会影响学习过程的，只是未必影响最终最优策略。

一般来说：

- 初始值偏大：会更乐观，可能鼓励探索
- 初始值偏小：会更保守
- 初值不同：会影响前期学习速度和探索行为

不过在你现在这个入门例子里，全 0 是最自然、最常见的选择。

10. 终止状态怎么理解

你说“到终点就不更新了”，这句话不完全准确。

更准确地说：

- 到达终止状态的那一步，通常仍然要更新当前这一步的 Q 值

- 只是终止之后已经没有后续动作了，所以“未来价值”这部分要特殊处理

也就是说，当 `next_state` 是终止状态时，可以理解为：

```
未来最优 Q 值 = 0
```

于是目标值就变成：

```
target = reward
```

在很多实现里，会写成类似：

```
if done:
    target = reward
else:
    target = reward + gamma * np.max(Q[next_state])
```

而你当前代码里：

```
target = reward + gamma * np.max(Q[next_state])
```

在 FrozenLake 里通常问题不大，因为终止状态那一行 Q 值往往本来就是 0。
但从概念上说，显式区分 `done` 会更严谨。

11. 训练和推理的区别

训练和推理本质上是在做两件不同的事：

- 训练：更新 Q 表，让估计越来越准
- 推理：使用训练好的 Q 表，按策略执行

所以：

- 训练时需要探索，允许随机动作
- 推理时通常直接选 `argmax(Q[state])`

你的 `FrozenLake.py` 是训练，
`FrozenLake_inference.py` 是推理，
这种分开写的方式很清楚。

12. Q-learning 和监督学习的区别

这是一个很适合写在笔记里的对比点。

监督学习通常是：

- 数据集已经给好了标准答案
- 模型的目标是尽量拟合标签

而 Q-learning 不是别人直接告诉你“标准 Q 值是多少”，而是：

- 智能体自己和环境交互
- 得到奖励和下一状态
- 再用
  `reward + gamma * max Q(next_state, :)`
  作为当前更新目标

所以 Q-learning 的目标值，不是人工标注出来的，而是“由当前估计和环境反馈共同构造出来的”。

这也是它和监督学习非常不同的地方。

13. 结合自己的代码理解一遍

在 `FrozenLake.py` 中，一次训练回合大致是：

1. 环境重置，得到初始状态
2. 用 `epsilon-greedy` 选择动作
3. 执行动作，得到 `next_state` 和 `reward`
4. 用更新公式修改 `Q[state, action]`
5. 转到下一个状态继续
6. 一个回合结束后衰减 `epsilon`

对应到代码里最关键的部分是：

```
if np.random.rand() < epsilon:
    action = env.action_space.sample()
else:
    action = greedy_action(Q, state)

next_state, reward, terminated, truncated, info = env.step(action)
done = terminated or truncated

old_value = Q[state, action]
target = reward + gamma * np.max(Q[next_state])
Q[state, action] = old_value + alpha * (target - old_value)
```

这个过程就是：

- 先试动作
- 再看结果
- 再把这次结果拿来修正当前动作值

14. 这阶段我最该记住的几句话

- `Q(s, a)` 不是当前奖励，而是“当前动作出发后的未来累计回报估计”
- Q-learning 学的是动作价值，不只是状态价值
- 当前 Q 值更新要看 `next_state` ，因为未来收益从下一状态开始决定
- 价值信息通常从终点附近开始，逐步向前传播
- `alpha` 控制更新幅度， `gamma` 控制看多远， `epsilon` 控制探索多少
- 训练是为了学 Q 表，推理是为了使用 Q 表

15. 我目前最容易记错的点

错误 1： `gamma = 0` 相当于没学

不对。

正确说法是：

`gamma = 0` 仍然在学，只是只学习即时奖励。

错误 2：不探索就完全不会更新

不对。

正确说法是：

不探索仍然会更新，但很容易只在很窄的路径上更新，学不到真正好的策略。

错误 3：到终点就不更新了

不对。

正确说法是：

到达终点的那一步仍然要更新，只是后续未来价值通常视为 0。

错误 4：Q 初值大小没影响

不完全对。

正确说法是：

Q 初值会影响前期学习速度和探索倾向，只是未必影响最终结果。

16. 之后还可以继续补充的内容

等我后面继续学强化学习时，可以把这篇笔记继续补成：

- Bellman equation 和 Q-learning 更新式的关系
- on-policy 和 off-policy 的区别
- Q-learning 和 SARSA 的区别
- 为什么连续状态下不能直接用 Q 表
- 从表格法过渡到 DQN